

ЛАБОРАТОРНА РОБОТА № 6

6.1 Тема

Робота з JSON, CSV та Excel файлами у Python

6.2 Мета роботи

Навчитися працювати з різними форматами файлів у Python (JSON, CSV, Excel), здійснювати читання, запис та маніпуляцію даними, а також використовувати відповідні бібліотеки (*json*, *csv*, *openpyxl*, *pandas*).

6.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- структуру та особливості форматів файлів JSON, CSV та Excel;
- основні методи та функції Python для роботи з файлами (*json.load*, *json.dump*, *csv.DictReader*, *csv.DictWriter*, *pandas.read_excel*, *DataFrame.to_excel*);
- принципи роботи з табличними та структурованими даними;
- правила обробки помилок при роботі з файлами.

2) уміти:

- зчитувати дані з JSON, CSV та Excel файлів;
- виконувати зміни в існуючих даних та додавати нові записи;
- зберігати змінені дані у нові файли у відповідному форматі;
- використовувати бібліотеки Python для обробки різних форматів файлів;
- відображати дані у зручному для користувача вигляді.

3) набути навичок:

- практичного опрацювання структурованих та табличних даних у Python;
- використання бібліотек *json*, *csv*, *pandas* для автоматизації обробки файлів;
- створення програмних рішень для обробки даних у різних форматах;
- організації даних у Python для подальшого аналізу та збереження у файли;

- безпечної та ефективної роботи з файлами з урахуванням можливих помилок.

6.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x;
- бібліотеки Python: json, csv, openpyxl, pandas (можна встановити через *pip install pandas openpyxl*).

6.5 Теоретичні відомості

У процесі розроблення прикладних програм часто виникає потреба зберігати, зчитувати та обробляти дані, що знаходяться у файлах різних форматів. Мова програмування Python надає широкий набір вбудованих засобів і бібліотек для роботи з файлами як у текстовому, так і в табличному вигляді. Найпоширенішими форматами обміну та зберігання даних є JSON, CSV та Excel.

Загальні принципи роботи з файлами у Python

Робота з файлами у Python здійснюється за допомогою функції `open()`, яка дозволяє відкривати файл у різних режимах:

- `r` – читання;
- `w` – запис (із перезаписом файлу);
- `a` – додавання даних;
- `r+`, `w+` – комбіновані режими.

Для коректної роботи з національними символами (українська мова) рекомендується використовувати параметр `encoding="utf-8"`.

Після завершення роботи з файлом його необхідно закрити. Найбільш безпечним способом є використання контекстного менеджера `with`, який автоматично закриває файл після завершення блоку коду.

Формат JSON та робота з ним у Python

JSON (JavaScript Object Notation) – це текстовий формат зберігання структурованих даних, який широко використовується для обміну інформацією між програмами та вебсервісами.

Основні особливості JSON:

- дані зберігаються у вигляді пар ключ–значення;
- підтримуються вкладені структури;
- легко відображається на типи даних Python.

Відповідність типів JSON і Python

JSON	Python
object	dict
array	list
string	str
number	int, float
true / false	True / False
null	None

Для роботи з JSON у Python використовується стандартний модуль `json`.

Основні функції модуля:

- `json.load(file)` – зчитування даних з JSON-файлу;
- `json.dump(data, file)` – запис даних у JSON-файл;
- `json.loads(string)` – перетворення JSON-рядка у структуру Python;
- `json.dumps(data)` – перетворення структури Python у JSON-рядок.

При записі даних часто використовують параметри:

- `ensure_ascii=False` – коректне збереження символів кирилиці;
- `indent=4` – форматований вивід для кращої читабельності.

Формат CSV та особливості його обробки

CSV (Comma Separated Values) – текстовий формат для зберігання табличних даних, у якому значення в одному рядку розділяються комами або іншими роздільниками.

Характерні риси CSV:

- простота структури;
- сумісність з електронними таблицями;
- зручність для зберігання табличних даних невеликого обсягу.

Для роботи з CSV у Python використовується стандартний модуль `csv`.

Основні класи та методи:

- `csv.reader()` – зчитування даних у вигляді списків;
- `csv.DictReader()` – зчитування у вигляді словників;
- `csv.writer()` – запис рядків у CSV-файл;
- `csv.DictWriter()` – запис словників у CSV-файл.

Перевагою використання `DictReader` та `DictWriter` є можливість звертатися до даних за назвами стовпців, що підвищує читабельність коду та зменшує ймовірність помилок.

Робота з Excel-файлами у Python

Формат Excel (`.xlsx`) широко використовується для зберігання та аналізу табличних даних. Для роботи з Excel у Python застосовуються зовнішні бібліотеки, зокрема `pandas` та `openpyxl`.

Бібліотека `pandas`

`pandas` – потужна бібліотека для аналізу даних, яка надає структуру `DataFrame` – двовимірну таблицю з рядками та стовпцями.

Основні можливості:

- зчитування даних з Excel (`read_excel`);
- фільтрація та сортування даних;
- додавання, видалення та зміна рядків і стовпців;
- запис результатів у Excel-файли (`to_excel`).

Бібліотека `openpyxl`

— `openpyxl` використовується як рушій для роботи з файлами `.xlsx`. Вона забезпечує фізичний доступ до Excel-файлів і дозволяє `pandas` коректно виконувати операції читання та запису.

Порівняння форматів JSON, CSV та Excel

Формат	Тип даних	Основне призначення
JSON	Структуровані дані	Обмін даними, збереження об'єктів
CSV	Табличні дані	Просте зберігання таблиць
Excel	Табличні дані	Аналіз, обробка, візуалізація

Вибір формату залежить від характеру даних, складності структури та вимог до подальшої обробки.

Обробка помилок при роботі з файлами

Під час роботи з файлами можливі помилки:

- файл не існує;
- відсутні права доступу;
- некоректний формат даних.

Для запобігання аварійному завершенню програми застосовують конструкцію `try-except`, яка дозволяє перехоплювати та обробляти винятки, пов'язані з файловими операціями.

Знання принципів роботи з форматами JSON, CSV та Excel є важливим етапом формування практичних навичок програмування мовою Python. Опанування цих засобів дозволяє створювати прикладні програми для зберігання, аналізу та обробки даних, що широко застосовуються в навчальній та професійній діяльності.

6.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.

- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

6.7 Загальне завдання

- 1) Встановити необхідні бібліотеки (openpyxl, pandas).
- 2) Створити структури даних у Python для роботи з файлами: словники, списки.
- 3) Виконати завдання з JSON:
 - зчитати дані з JSON-файлу;
 - обробити дані (додати, змінити, видалити записи);
 - записати змінені дані у новий JSON-файл.
- 4) Виконати завдання з CSV:
 - зчитати дані з CSV-файлу у список або DataFrame;
 - змінити дані, додати або видалити рядки;
 - зберегти результат у новий CSV-файл.
- 5) Виконати завдання з Excel:
 - зчитати дані з Excel-файлу у DataFrame;
 - виконати обробку даних (фільтрація, сортування);
 - записати оброблені дані у новий Excel-файл.

6.8 Індивідуальні завдання

- 1) Створити програму для зчитування інформації про студентів (ім'я, вік, факультет) із JSON-файлу та виведення на екран.
- 2) Розробити програму для додавання нового студента до списку студентів у JSON-файлі.
- 3) Написати програму для пошуку студента за ім'ям у JSON-файлі та виведення його даних.

- 4) Розробити програму для зміни інформації про студента в JSON-файлі (наприклад, зміна факультету).
- 5) Написати програму для видалення студента з JSON-файлу за певним іменем.
- 6) Створити програму для сортування студентів за віком і запису результату у новий JSON-файл.
- 7) Розробити програму для підрахунку середнього віку студентів і запису результату у JSON.
- 8) Написати програму для створення JSON-файлу з інформацією про курси, які викладаються на факультеті.
- 9) Розробити програму для конвертації даних з формату CSV у JSON і збереження результату в JSON-файл.
- 10) Створити програму для порівняння двох JSON-файлів та виведення різниць між ними.

Нульовий варіант (приклад виконання завдання з JSON-файлом)

Створити програму, яка демонструє роботу з JSON-файлом. Програма повинна:

- 1) Зчитати дані про студентів із JSON-файлу (students.json).
- 2) Вивести список студентів на екран.
- 3) Додати нового студента до списку.
- 4) Зберегти оновлені дані у новий JSON-файл (students_updated.json).

JSON-файл (students.json) – приклад вмісту:

```
[
  {"name": "Іван", "age": 20, "faculty": "ФІТ"},
  {"name": "Марія", "age": 19, "faculty": "Економіка"},
  {"name": "Олег", "age": 21, "faculty": "Мехатроніка"}
]
```

```
# Підключаємо модуль для роботи з JSON
import json

# 1. Зчитування даних з JSON-файлу
with open("students.json", "r", encoding="utf-8") as file:
    students = json.load(file) # JSON перетворюється на список словників

# 2. Вивід даних на екран
```

Основи розробки програмного забезпечення мовою програмування Python.

к.т.н., доц. Барбарук Віктор Миколайович

```

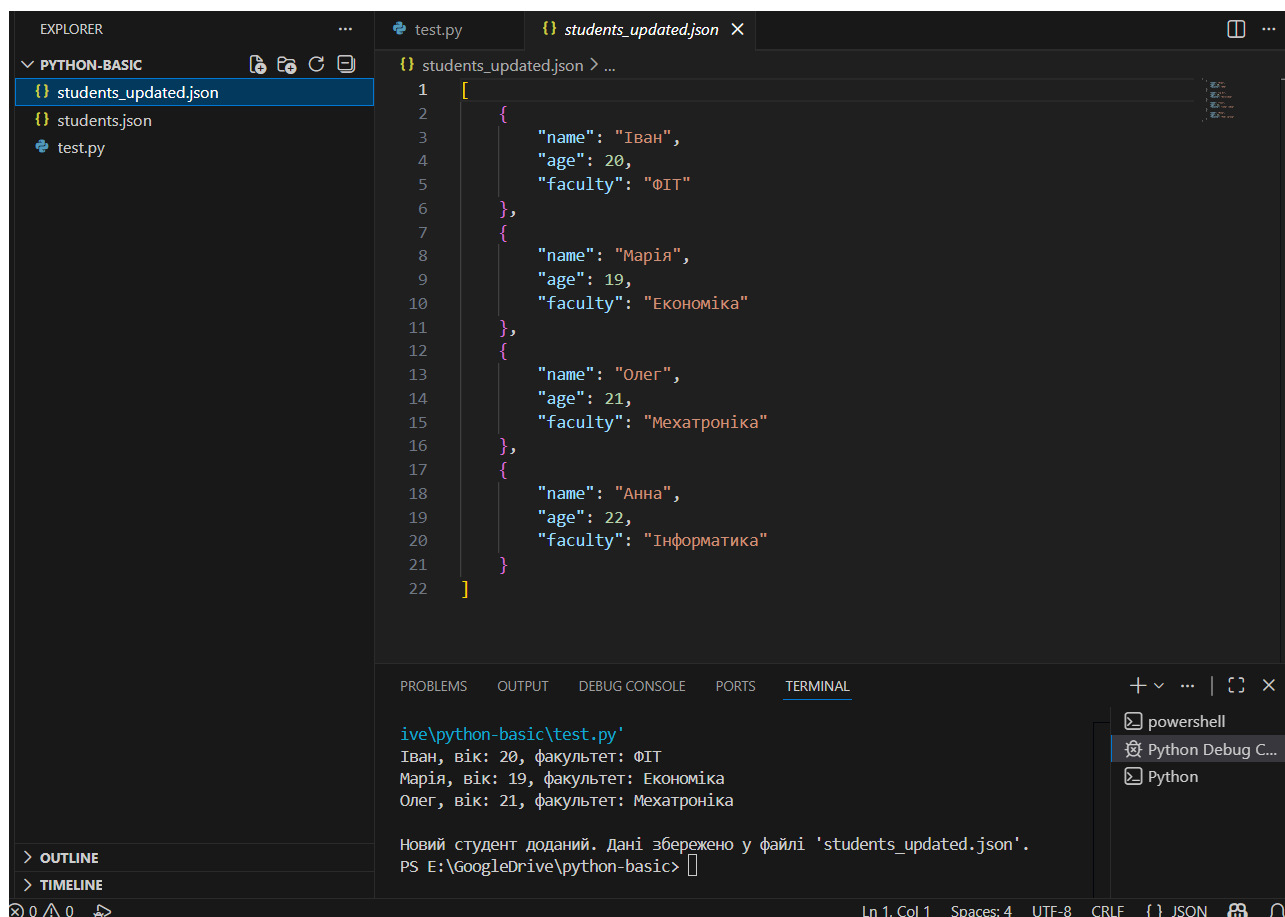
print("Поточний список студентів:")
for student in students:
    print(f"{student['name']}, вік: {student['age']}, факультет: {student['faculty']}")

# 3. Додавання нового студента
new_student = {
    "name": "Анна",
    "age": 22,
    "faculty": "Інформатика"
}
students.append(new_student)

# 4. Запис оновлених даних у новий файл
with open("students_updated.json", "w", encoding="utf-8") as file:
    json.dump(students, file, ensure_ascii=False, indent=4)

print("\nНовий студент доданий. Дані збережено у файлі 'students_updated.json'.")

```



Нульовий варіант (приклад виконання завдання з CSV-файлом)

Завдання:

- 1) Зчитати дані про студентів із CSV-файлу (students.csv).
- 2) Вивести список студентів на екран.

Основи розробки програмного забезпечення мовою програмування Python.

к.т.н., доц. Барбарук Віктор Миколайович

3) Додати нового студента до списку.

4) Зберегти оновлені дані у новий CSV-файл (students_updated.csv).

CSV-файл (students.csv) – приклад вмісту:

```
name,age,faculty
Іван,20,ФІТ
Марія,19,Економіка
Олег,21,Мехатроніка
```

Приклад реалізації

```
import csv

# 1. Зчитування даних з CSV-файлу
students = []
with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file) # Зчитуємо у форматі словників
    for row in reader:
        # Конвертуємо вік у число
        row['age'] = int(row['age'])
        students.append(row)

# 2. Вивід даних на екран
print("Поточний список студентів:")
for student in students:
    print(f"{student['name']}, вік: {student['age']}, факультет: {student['faculty']}")

# 3. Додавання нового студента
new_student = {
    "name": "Анна",
    "age": 22,
    "faculty": "Інформатика"
}
students.append(new_student)

# 4. Запис оновлених даних у новий CSV-файл
with open("students_updated.csv", "w", newline="", encoding="utf-8") as file:
    fieldnames = ["name", "age", "faculty"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader() # Записуємо заголовки
    writer.writerows(students) # Записуємо всі рядки

print("\nНовий студент доданий. Дані збережено у файлі 'students_updated.csv'.")
```

Результат роботи

The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree showing a project named 'PYTHON-BASIC' containing files: `students_updated.csv`, `students_updated.json`, `students.csv`, `students.json`, and `test.py`.
- Editor:** The file `students_updated.csv` is open, showing a table with 6 rows and 3 columns: `name`, `age`, and `faculty`. The data is as follows:

name	age	faculty
Іван	20	ФІТ
Марія	19	Економіка
Олег	21	Мехатроніка
Анна	22	Інформатика
- TERMINAL:** The terminal shows the output of a Python script:


```
ive\python-basic\test.py'
Поточний список студентів:
Іван, вік: 20, факультет: ФІТ
Марія, вік: 19, факультет: Економіка
Олег, вік: 21, факультет: Мехатроніка

Новий студент доданий. Дані збережено у файлі 'students_updated.csv'.
PS E:\GoogleDrive\python-basic>
```

Нульовий варіант (приклад виконання завдання з Excel-файлом)

Завдання:

- 1) Зчитати дані про студентів із Excel-файлу (`students.xlsx`).
- 2) Вивести список студентів на екран.
- 3) Додати нового студента до списку.
- 4) Зберегти оновлені дані у новий Excel-файл (`students_updated.xlsx`).

Excel-файл (`students.xlsx`):

- Лист: Sheet1
- Колонки: `name`, `age`, `faculty`
- Дані: ті ж, що у CSV

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	name	age	faculty										
2	Іван	20	ФІТ										
3	Марія	19	Економіка										
4	Олег	21	Мехатроніка										
5													
6													
7													

Приклад реалізації

```
import pandas as pd

# 1. Зчитування даних з Excel-файлу
students_df = pd.read_excel("students.xlsx")

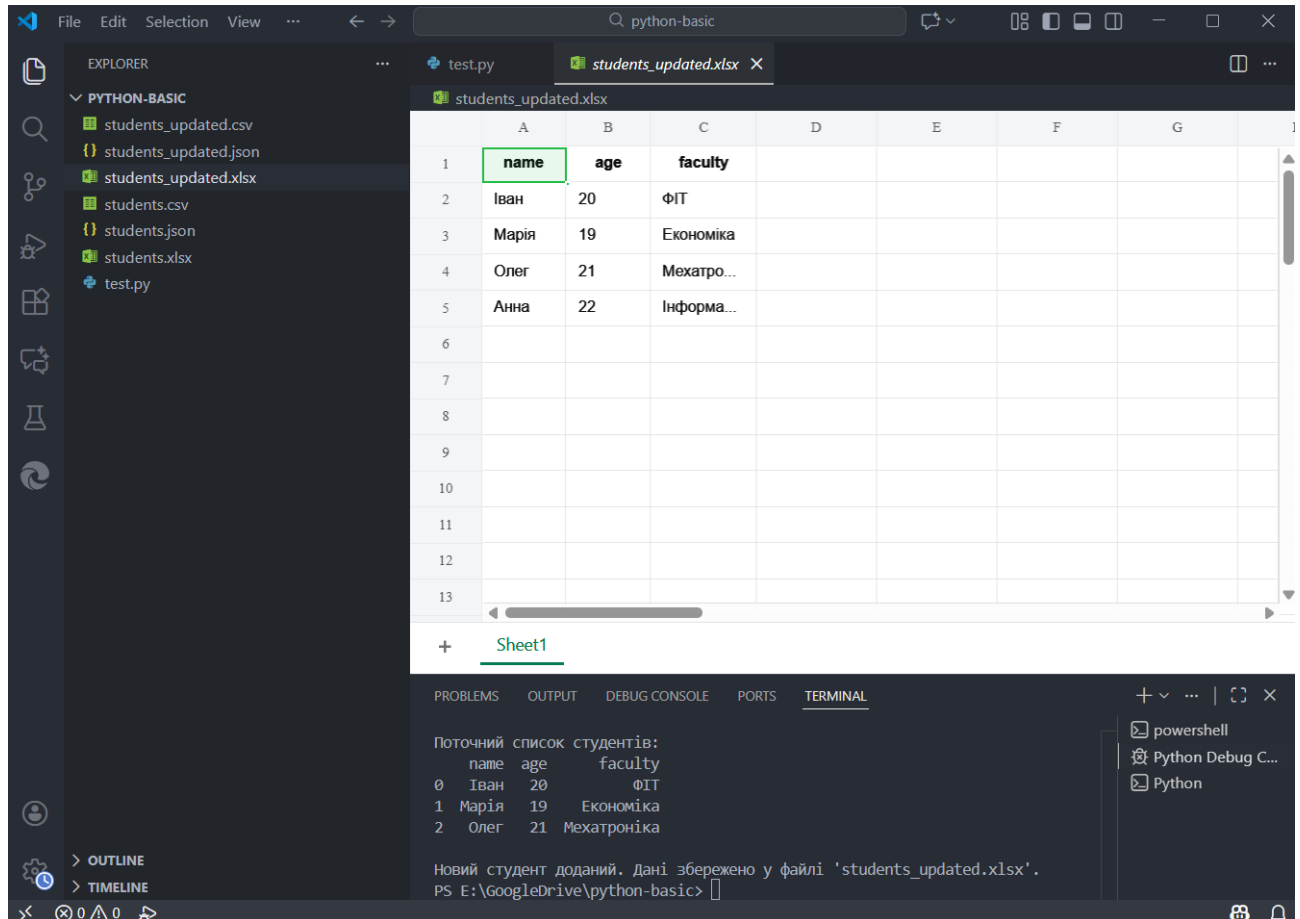
# 2. Вивід даних на екран
print("Поточний список студентів:")
print(students_df)

# 3. Додавання нового студента
new_student = {"name": "Анна", "age": 22, "faculty": "Інформатика"}
students_df = pd.concat([students_df, pd.DataFrame([new_student])],
                        ignore_index=True)

# 4. Запис оновлених даних у новий Excel-файл
students_df.to_excel("students_updated.xlsx", index=False)

print("\nНовий студент доданий. Дані збережено у файлі 'students_updated.xlsx'.")
```

Результати роботи



Пояснення коду Excel / CSV:

- 1) CSV: використовується модуль csv для роботи зі словниками (DictReader, DictWriter)
- 2) Excel: використовується pandas для зручного зчитування та запису таблиць у форматі .xlsx
- 3) В обох варіантах демонструється стандартний потік роботи: зчитування → вивід → зміни → запис
- 4) Код можна брати за нульовий шаблон і адаптувати для індивідуальних варіантів завдань

6.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.

- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

6.10 Контрольні питання

- 1) Як зчитати дані з JSON-файлу за допомогою Python?
- 2) Що таке структура JSON і як вона співвідноситься з типами даних у Python?
- 3) Як додавати нові елементи в існуючий JSON-файл?
- 4) Які методи Python використовуються для запису змін у JSON-файл?
- 5) Як можна обробляти вкладені структури (списки, словники) в JSON?

6.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>